VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Internet of Things

Project Report

IoT Project Report

Class TN01– Group HT– Semester 252

Advisor(s): Lê Trọng Nhân && Phan Văn Sỹ

Student(s): Nguyễn Việt Hoàng 2311066

Nguyễn Chí Thanh 2313078

HO CHI MINH CITY, MAY 2026

Abstract

This report presents the design and implementation of a real-time Internet of Things system on the ESP32-S3 platform using FreeRTOS and PlatformIO. The system was developed from an RTOS-based starter project and extended into a multi-functional environmental monitoring node that combines local sensing, real-time visual feedback, a web-based user interface, on-device TinyML prediction, and CoreIOT cloud connectivity. The final implementation is not a single-purpose sensor demo; instead, it is a coordinated embedded system in which independent tasks exchange data through queues, semaphores, and mutexes.

The implemented node periodically acquires temperature and humidity from a DHT20 sensor, drives a discrete LED according to temperature conditions, updates a NeoPixel according to humidity level, and renders context-aware information on a 16x2 I^2C LCD. In parallel, a web server supports both Access Point (AP) and Station (STA) operation. In AP mode, the board exposes a dashboard for live monitoring, Wi-Fi scanning, saved-profile management, and direct control of an external LED through on/off and brightness-adjustment actions. In STA mode, the node connects to the Internet and publishes telemetry to the CoreIOT platform using MQTT. The project also includes a TinyML task that performs batched inference on normalized temperature-humidity features and generates a weather-oriented rain/no-rain prediction.

From a software-engineering perspective, the project emphasizes task separation, predictable timing, and reduced dependence on globally shared state. Sensor data are treated as streaming information and distributed through single-slot queues to the LCD, web, TinyML, and cloud modules. An application context structure stores shared synchronization objects and a compact shared-state block protected by a mutex. This approach keeps the overall architecture modular while preserving the current logic of the system.

The report explains the assignment requirements, maps them to the implemented features, describes the task architecture and data flow, and analyzes the timing behavior of each subsystem. Based on the code structure and diagnostic logs, the implementation satisfies the assignment as a soft real-time IoT system. It achieves periodic sensing, event-driven user interaction, Wi-Fi mode switching, lightweight on-device inference, and successful preparation for cloud telemetry publishing in STA mode.

Contents

Abstract **i**

1 Introduction **1**

1.1 Source Code . . . 1
1.2 Project Context . . . 1
1.3 Assignment Tasks and Design Implications . . . 1
1.4 From Requirements to System Construction . . . 3
1.5 Project Objectives . . . 4
1.6 Contributions of the Implemented System . . . 4

2 Requirements **5**

2.1 Requirement Mapping Strategy . . . 5
2.2 Task 1 Requirements: Temperature-Dependent LED Behavior . . . 5
2.3 Task 2 Requirements: Humidity-Dependent NeoPixel Indication . . . 6
2.4 Task 3 Requirements: LCD-Based Monitoring and RTOS Synchronization . . . 6
2.5 Task 4 Requirements: AP-Mode Web Server and User Interaction . . . 7
2.6 Task 5 Requirements: TinyML Deployment and Evaluation . . . 7
2.7 Task 6 Requirements: CoreIOT Cloud Publishing . . . 8
2.8 Cross-Cutting Functional Requirements . . . 9
2.9 Non-Functional Requirements . . . 9
2.10 Hardware and Software Environment . . . 10

3 Design **10**

3.1 Design Goals . . . 10
3.2 System Architecture Overview . . . 11
3.3 Hardware Mapping . . . 11
3.4 RTOS Task Architecture . . . 12
3.5 Application Context and Shared State . . . 14
3.6 Inter-Task Communication Model . . . 14
3.7 Queue Topology . . . 15
3.8 State Flow Through the System . . . 16
3.9 Design Rationale . . . 17

4 Implementation **17**

4.1 System Startup and Resource Allocation . . . 17

4.2 Shared State Helpers . . . 18
4.3 Sensor Task . . . 18
4.4 LED Manager Task . . . 19
4.5 NeoPixel Task . . . 20
4.6 LCD Task . . . 20
4.7 Web Server Task . . . 21
4.8 TinyML Task: Edge-Based Rain Forecasting Pipeline . . . 24
4.9 CoreIOT Task: Cloud Telemetry and Remote Configuration . . . 32
4.10 MQTT Broker Implementation: TinyMQTT Service . . . 36
4.11 Time Utility Module . . . 42
4.12 Implementation Qualities . . . 43

5 Testing **44**
5.1 Testing Methodology . . . 44
5.2 Test Plan . . . 44
5.3 Representative Log Evidence . . . 44
5.4 Timing Analysis by Task . . . 45
5.5 Evaluation of RTOS Timing Quality . . . 45
5.6 Task-by-Task Functional Verification . . . 47
5.7 End-to-End Responsiveness . . . 48
5.8 Current Limitations of the Test Evidence . . . 49

6 Results and Discussion **49**
6.1 What Works Well . . . 49
6.2 Tradeoffs in the Current Design . . . 50
6.3 Current Weaknesses . . . 51
6.4 Future Development Directions . . . 52

7 Conclusion and Future Work **52**
7.1 Conclusion . . . 52
7.2 What Was Learned . . . 53
7.3 Future Work . . . 53

1 Introduction

1.1 Source Code

The source code for this project is available in a public GitHub repository at our github. The repository contains the complete implementation of the ESP32-S3 application, including all source files, configuration files, and documentation.

1.2 Project Context

The objective of this project is to transform a baseline RTOS-based embedded application into a complete Internet of Things system on the ESP32-S3 platform. The target board provides sufficient GPIO, Wi-Fi connectivity, and compute resources to host multiple concurrent tasks under FreeRTOS while still keeping the implementation close to the constraints of an embedded device. The project is built with PlatformIO and the Arduino framework, but its internal organization follows a classical RTOS design with explicitly created tasks, queues, semaphores, and mutexes rather than a monolithic `loop()` application. This is important because the assignment is not only about adding features; it also requires students to demonstrate understanding of task coordination, modularity, and synchronization in an embedded real-time system [**freertos_kernel**, **platformio**, **esp32_arduino**].

The final system can be understood as a smart environmental node. It reads temperature and humidity, presents the state locally through LEDs and an LCD, offers a browser-based configuration and monitoring interface, performs on-device TinyML inference, and publishes telemetry to a cloud platform. Compared to a simple sensor board, the project therefore integrates four different interaction layers:

1. the physical sensing layer (DHT20, LCD, LEDs, NeoPixel),
2. the RTOS coordination layer (tasks, queues, semaphores, shared state),
3. the local network user-interface layer (AP/STA web server), and
4. the cloud-connected telemetry layer (CoreIOT via MQTT).

1.3 Assignment Tasks and Design Implications

The assignment defines six major tasks. Understanding the constraints of each task is essential because the final architecture must support all of them simultaneously instead of implementing them in isolation.

Task 1: Single LED blink with temperature conditions.

The first requirement is to redefine the blink behavior of a single LED according to at least three temperature states. This requirement looks simple, but it introduces two important design issues. First, the LED task must not block sensor acquisition or networking. Second, LED mode changes should happen immediately when a new temperature value arrives. This naturally suggests a dedicated LED manager task that receives temperature-derived commands through a queue.

Task 2: NeoPixel control based on humidity.

The second task requires humidity-dependent color feedback. In this project, the NeoPixel is not used as a static indicator but as a continuous color mapper from low to high humidity. This expands the assignment requirement into a more expressive visual representation. Because humidity values are produced by the sensor task and consumed asynchronously by the NeoPixel task, queue-based data delivery is again the most appropriate mechanism.

Task 3: Temperature and humidity monitoring with LCD display.

The LCD requirement is more subtle than it first appears. The display must show environmental information, react to system state, and demonstrate RTOS synchronization. In the implemented system, the LCD is not limited to one screen. It supports sensor display, Wi-Fi status display, and TinyML forecast display. The task also forces correct handling of a shared I^2C bus, which is why the code uses a mutex to protect both the DHT20 and LCD access.

Task 4: Web server in Access Point mode.

The web server task introduces a complete user-interface requirement on top of the embedded node. The board must expose a meaningful AP-mode interface and allow control of two actions. In this project, the AP dashboard not only shows live sensor data but also supports Wi-Fi scanning, Wi-Fi profile storage, AP to STA switching, external LED on/off control, and external LED brightness control. This task directly influences the architecture because the web task must remain responsive while other RTOS tasks continue to run periodically.

Task 5: TinyML deployment and accuracy evaluation.

This task requires more than simply executing a model. The system must define a data pipeline, prepare model inputs, deploy inference on the ESP32-S3, and evaluate the result. In the current

implementation, TinyML is integrated as a dedicated consumer task that accumulates a batch of sensor readings, derives eight normalized input features, invokes a TensorFlow Lite Micro model, and publishes the result to the rest of the system. This requirement has a strong architectural impact because inference is periodic, compute-intensive relative to the rest of the application, and semantically different from raw sensing.

Task 6: Data publishing to CoreIOT cloud server.

Cloud integration introduces the requirement that the board must publish data in STA mode only, with the correct authentication token. In practice, this means the application needs a mechanism to detect when Wi-Fi has successfully moved from AP or disconnected state into valid STA connectivity. This project implements that transition with a binary semaphore that signals "Internet ready" from the web task to the CoreIOT task. The cloud task therefore does not need to continuously probe for connectivity from boot time.

1.4 From Requirements to System Construction

Taken together, the six assignment tasks strongly suggest a distributed task architecture rather than a sequential application. A single-threaded design would quickly become difficult to maintain, because sensor acquisition, LCD rendering, blink control, Wi-Fi handling, HTTP requests, TinyML inference, and MQTT publishing all have different execution patterns and timing expectations.

For this reason, the implemented system is built around seven FreeRTOS tasks: `sensor_task`, `led_manager_task`, `neo_pixel_task`, `lcd_task`, `main_server_task`, `coreiot_task`, and `tiny_ml_task`. The main function only allocates an application context, creates queues and synchronization primitives, and starts these tasks. Each task is intentionally narrow in responsibility: the sensor task owns periodic sensing, the LED and NeoPixel tasks own local actuation, the LCD task owns presentation, the web task owns UI and Wi-Fi mode management, the TinyML task owns prediction, and the CoreIOT task owns cloud communication.

This decomposition also makes the system easier to explain and test. Each assignment task can be traced to one or more RTOS tasks and their data interfaces. For example, the sensor task produces the physical environment measurements, the LED and NeoPixel tasks consume them to fulfill Tasks 1 and 2, the LCD and web tasks consume them to fulfill Tasks 3 and 4, the TinyML task transforms them to fulfill Task 5, and the CoreIOT task publishes them to fulfill Task 6.

1.5 Project Objectives

Based on the assignment and the implemented architecture, the practical objectives of this project are the following:

- to build a stable multi-task IoT application on ESP32-S3;
- to demonstrate correct use of FreeRTOS queues, semaphores, and mutexes;
- to implement meaningful local feedback for temperature and humidity;
- to provide a usable AP-mode dashboard for monitoring and control;
- to support STA-mode cloud publishing to CoreIOT;
- to integrate TinyML inference into the live sensing pipeline; and
- to keep the design modular enough for future extension.

1.6 Contributions of the Implemented System

Compared with a minimal RTOS demonstration, the current system contributes the following integrated features:

- periodic sensing every 2 seconds using a dedicated sensor task;
- temperature-dependent LED blinking with multiple operating modes;
- humidity-dependent NeoPixel color mapping;
- a multi-mode LCD that adapts to AP, STA, and TinyML states;
- a browser-accessible AP dashboard with charting and device control;
- Wi-Fi scanning, profile storage, and automatic reconnection flow;
- batched TinyML inference using an 8-feature input vector;
- CoreIOT MQTT telemetry publishing every 10 seconds in STA mode; and
- a shared application context that reduces the need for project-wide global variables.

The remainder of this report explains how these features map to the assignment, how they are implemented in code, and how the behavior of each task can be analyzed from a system-design and RTOS perspective.

2 Requirements

2.1 Requirement Mapping Strategy

The assignment statement provides high-level tasks, but a report must explain how those tasks become concrete, testable requirements in the implemented system. In this project, each assignment task is refined into two layers:

1. a functional requirement that can be observed by the user, and
2. an implementation-oriented requirement that constrains timing, task communication, or system architecture.

This section follows the same logic. It first describes the detailed requirements of each task and then summarizes the non-functional properties that the complete system must satisfy.

2.2 Task 1 Requirements: Temperature-Dependent LED Behavior

The assignment requires at least three distinct LED behaviors based on temperature. In the implemented system, the following concrete requirements were derived:

- The system shall classify temperature into at least three operating regions.
- The discrete LED shall blink with a different cadence for each temperature region.
- LED mode changes shall be triggered automatically whenever a new valid temperature sample is produced.
- The LED logic shall run independently of the sensor task so that the sensing period remains stable.
- The LED task shall also support short event pulses for system feedback such as Wi-Fi-connected or error-notification events.

The current implementation satisfies these requirements by mapping temperature to three primary modes: cold, normal, and hot. These modes correspond to significantly different blink half-periods, which makes the environmental state easy to distinguish even without reading the LCD or web dashboard.

2.3 Task 2 Requirements: Humidity-Dependent NeoPixel Indication

For the NeoPixel task, the assignment requires humidity-based color indication using semaphore-style synchronization. In the implemented project, the requirement was refined as follows:

- The system shall compute a humidity-dependent color output for the NeoPixel.
- The color shall reflect at least three semantic humidity regions, even if the implementation internally uses a continuous gradient.
- The NeoPixel update shall be decoupled from sensing through a queue so that the LED strip always displays the most recent humidity state.
- The NeoPixel task shall support an enable/disable command and clear the LED when disabled.

In the current code, humidity values are constrained to the range 0% to 100% and converted into a color gradient. The lower half of the range is represented by a transition from red to green, while the upper half is represented by a transition from green to blue. This means the assignment's "at least three levels" condition is exceeded by a smooth visual scale.

2.4 Task 3 Requirements: LCD-Based Monitoring and RTOS Synchronization

The LCD task was extended beyond a static two-line text display. The refined requirements are:

- The LCD shall display valid temperature and humidity measurements when the sensor is operating normally.
- The LCD shall display an error message when sensing fails.
- The LCD shall support more than one display state, including a sensor mode and a network-aware mode.
- In STA mode, the LCD shall be able to switch between raw sensor data and TinyML forecast information.
- The DHT20 sensor and LCD share the same I^2C bus, so access to that bus shall be protected by a mutex.

The implemented design therefore uses the LCD not only as an output device but also as a demonstration of synchronization. The I^2C mutex is one of the clearest examples in the code where a shared hardware resource is explicitly protected.

2.5 Task 4 Requirements: AP-Mode Web Server and User Interaction

The web server task is one of the most visible parts of the project and also one of the most functionally rich. The concrete requirements implemented in the current version are:

- The board shall provide an HTTP server when running in AP mode.
- The root dashboard shall show temperature, humidity, TinyML prediction, Wi-Fi state, and chart history.
- The web server shall provide Wi-Fi scanning so the user can list available networks.
- The web server shall provide storage and recall of saved Wi-Fi credentials.
- The interface shall provide at least two direct control actions. In this project those actions are:
  - external LED on/off toggle,
  - external LED brightness adjustment through a PWM slider.
- The system shall be able to move from AP mode to STA mode using browser-supplied SSID and password.
- If STA connection does not succeed within the timeout window, the system shall return to AP mode automatically.
- Pressing the BOOT button shall force the board back to AP mode.

The web task therefore covers both UI and network-state control. This is more than a simple monitoring page: it acts as the configuration front-end for the entire device.

2.6 Task 5 Requirements: TinyML Deployment and Evaluation

The TinyML portion of the assignment requires a complete mini-pipeline from data representation to inference. The concrete requirements implemented in code are:

- The system shall deploy a TensorFlow Lite Micro model on the ESP32-S3.

- The TinyML task shall receive live sensor data from the RTOS pipeline.
- Input features shall be transformed and normalized before inference.
- The model output shall be converted into an interpretable prediction.
- The prediction result shall be accessible to other tasks, especially the LCD task, the web task, and the CoreIOT task.
- The report shall discuss the expected inference timing and the impact of batching and feature engineering on accuracy and responsiveness.

The implemented TinyML logic uses batch averaging and derived time-based features rather than passing raw sensor samples directly to the model. This is important because it shows that TinyML integration is treated as a system design problem, not merely as a call to an inference library [**tflm**].

2.7 Task 6 Requirements: CoreIOT Cloud Publishing

The cloud task is constrained by the assignment's STA-mode rule. The refined requirements are:

- The board shall connect to CoreIOT only when Wi-Fi STA mode is active and Internet access is available.
- The device shall use a valid CoreIOT token to authenticate.
- Telemetry shall include temperature and humidity, and may include additional derived fields that improve observability.
- The cloud task shall periodically publish data without blocking local sensing and control.
- The system shall subscribe to RPC messages so that a remote cloud command can influence local behavior.

In the current implementation, the RPC channel is used to control whether the LCD should show the TinyML forecast in STA mode. This is a strong design choice because it demonstrates bidirectional interaction with the cloud rather than simple telemetry upload [**coreiot_platform**, **pubsubclient**, **arduinojson**].

2.8 Cross-Cutting Functional Requirements

Beyond the six assignment tasks, the following system-level functional requirements emerge from the integrated design:

- The latest sensor sample shall be distributed to multiple consumers without unnecessary duplication of acquisition logic.
- The user shall be able to understand the state of the system through local logs, LCD messages, and browser-visible status labels.
- The system shall preserve the most recent Wi-Fi profiles across resets by using non-volatile storage.
- The dashboard shall plot recent sensor history rather than showing only the most recent sample.
- The cloud payload shall include both raw values and interpreted state such as rain probability and LCD forecast mode.

2.9 Non-Functional Requirements

The project also has important non-functional constraints:

- **Real-time structure:** sensing, control, display, networking, inference, and cloud publishing must coexist without unacceptable blocking.
- **Responsiveness:** web interactions should feel immediate to a human user, while periodic tasks should keep predictable cadence.
- **Modularity:** different subsystems should be readable and maintainable as separate source files and tasks.
- **Reduced shared global state:** the project should avoid broad global-variable usage and instead favor queues, semaphores, and protected shared state.
- **Fault tolerance:** the system should tolerate Wi-Fi disconnects, STA connection failure, and sensor read failure without total application collapse.
- **Scalability:** the architecture should be extendable with more sensors, more RPC methods, or richer dashboards in the future.

2.10 Hardware and Software Environment

The current implementation is designed around the following hardware and software stack:

- **Board:** ESP32-S3-based Yolo UNO platform.
- **Sensor:** DHT20 temperature and humidity sensor on the I^2C bus.
- **LCD:** 16x2 I^2C LCD at address `0x21`.
- **Discrete LED:** GPIO48 for temperature-dependent blink logic.
- **NeoPixel:** GPIO45, one RGB LED for humidity visualization.
- **External user-controlled LED:** GPIO5 with PWM control from the web interface.
- **Build environment:** PlatformIO with Arduino framework.
- **Software components:** FreeRTOS primitives, WebServer, Preferences, PubSubClient, ArduinoJson, and TensorFlow Lite Micro [**platformio**, **freertos_kernel**, **pubsubclient**, **arduinojson**, **tflm**].

These hardware and software choices define the practical limits of the system and shape the architecture described in the next section.

3 Design

3.1 Design Goals

The design of the system follows one central principle: each major responsibility should be owned by one RTOS task, and communication between tasks should happen through explicit synchronization primitives rather than implicit shared-state dependencies. This is especially important for this project because the system combines periodic sensing, asynchronous user interaction, local rendering, network-state transitions, TinyML inference, and MQTT communication. If these concerns were implemented in a single control loop, the result would be harder to reason about and harder to evaluate as an RTOS design.

The architecture therefore aims to satisfy five goals:

1. predictable periodic sensing;
2. clear separation between producers and consumers of data;
3. minimal blocking between unrelated modules;

4. controlled access to shared resources such as the I^2C bus and shared state; and
5. graceful transition between AP mode, STA mode, local UI, and cloud connectivity.

3.2 System Architecture Overview

At the highest level, the system can be viewed as a streaming pipeline centered around sensor acquisition. The `sensor_task` periodically produces the latest environmental measurements and distributes them to the rest of the system. Several downstream tasks consume the same information for different purposes:

- the LED task converts temperature into blink behavior,
- the NeoPixel task converts humidity into color,
- the LCD task renders local status,
- the web task updates the browser-visible dashboard and history buffer,
- the TinyML task performs prediction, and
- the CoreIOT task publishes telemetry to the cloud.

The web task is also the owner of Wi-Fi mode management. It decides whether the board is running in AP mode or attempting STA connection, and it signals the CoreIOT task when Internet connectivity becomes available. The TinyML and CoreIOT tasks are therefore not directly responsible for network setup; they are consumers of network availability.

3.3 Hardware Mapping

Table 3.1 summarizes the active hardware resources used by the project.

Table 3.1: Hardware mapping of the implemented system

Component	**Pin / Address**	**Purpose**
ESP32-S3 board	–	Main microcontroller, Wi-Fi connectivity, RTOS execution
DHT20 sensor	I^2C bus	Temperature and humidity acquisition
LCD 16x2	I^2C address `0x21`	Local system display for sensor, Wi-Fi, and TinyML states
I^2C SDA	GPIO11	Shared bus data line
I^2C SCL	GPIO12	Shared bus clock line
Discrete LED	GPIO48	Temperature-dependent blink feedback
NeoPixel	GPIO45	Humidity-dependent color indication
External LED	GPIO5	User-controlled LED from the web interface, PWM brightness
BOOT button	GPIO0	Manual fallback from STA/AP flow back to AP mode

The mapping shows that the architecture is constrained by both physical sharing and functional reuse. The DHT20 and LCD share the I^2C bus, which justifies the use of a mutex. The external LED is distinct from the temperature LED, which makes it possible to satisfy the web control task without interfering with the environmental blink behavior.

3.4 RTOS Task Architecture

The complete runtime is built from seven explicitly created tasks. Their responsibilities, priorities, and triggering models are shown in Table 3.2.

Table 3.2: FreeRTOS task architecture

Task	Prio.	Stack	Activation model	Main responsibility
`sensor_task`	3	4096	Periodic every 2 s	Read DHT20, validate values, dispatch samples to downstream consumers
`lcd_task`	2	4096	Event-driven, queue wait up to 250 ms	Render local LCD content depending on sensor, Wi-Fi, and TinyML state
`main_server_task`	4	8192	Fast service loop every 20 ms	Run HTTP server, handle AP/STA state, Wi-Fi profiles, dashboard logic
`led_manager_task`	1	3072	Event-driven plus timeout	Generate discrete LED blink patterns from temperature or event commands
`neo_pixel_task`	1	3072	Event-driven, blocking queue receive	Map humidity to RGB color and update NeoPixel
`tiny_ml_task`	1	8192	Event-driven by sensor queue	Accumulate samples, build features, invoke TFLM model, publish result
`coreiot_task`	1	6144	Event-driven after Internet semaphore + periodic publish	Connect MQTT client, serve RPC, publish telemetry every 10 s

This division is a good fit for the project requirements. The web task receives the highest priority because it is responsible for configuration and network mode transitions. The sensor task receives the next-highest priority because it defines the cadence of environmental sampling. Presentation and cloud tasks are lower priority because they consume already acquired data rather than producing the primary physical measurements.

3.5 Application Context and Shared State

The project uses an `app_context_t` structure as the central ownership object for all queues, mutexes, the Internet-ready semaphore, and a compact shared state block. This is an important architectural decision because it reduces the need for project-wide global variables. Instead of exposing individual globals for temperature, humidity, Wi-Fi connection state, or LCD content mode, the code stores them inside a shared state structure protected by `stateMutex`.

The shared state currently contains:

- latest temperature,
- latest humidity,
- a boolean Wi-Fi-connected flag, and
- the current LCD content mode.

This does not remove every file-local static variable from the repository. The web server module, for example, still maintains private runtime variables for its internal HTML state, history buffer, and Wi-Fi workflow. However, it significantly reduces uncontrolled cross-module coupling and improves the clarity of synchronization.

3.6 Inter-Task Communication Model

The project uses three primary synchronization mechanisms.

Queues.

Queues are used for directional data flow. The design intentionally favors single-slot queues for continuously refreshed information. This means the system does not attempt to preserve every historical sensor sample inside RTOS queues; instead, each consumer receives the most recent snapshot. This choice is appropriate for UI and cloud applications where stale data are less useful than fresh data.

Mutexes.

Two mutexes are central to the design:

- `i2cMutex` protects the shared I^2C bus between DHT20 reads and LCD writes.
- `stateMutex` protects access to the shared state block.

Without `i2cMutex`, concurrent sensor and LCD access could lead to corrupted transactions or inconsistent device behavior. Without `stateMutex`, Wi-Fi or LCD content state could be modified while another task reads it.

Binary semaphore.

The `internetSemaphore` is used as a one-shot signal from the web task to the CoreIOT task. It encodes an event rather than a data value: "STA connectivity is now ready for cloud communication." This is an elegant use of a binary semaphore because the CoreIOT task should not try to initialize MQTT before the device is in the correct network mode.

3.7 Queue Topology

The queue topology is intentionally asymmetric. The sensor task is the main producer and fans out data into multiple destinations. Table 3.3 shows the purpose of each queue.

Table 3.3: Queue topology and semantics

Queue	Length	Producer	Consumer / Purpose
`ledQueue`	8	sensor task or system events	LED manager receives mode-change and pulse commands
`neoQueue`	1	sensor task	NeoPixel receives latest humidity snapshot
`lcdQueue`	1	sensor task	LCD receives latest environmental sample
`webQueue`	1	sensor task	Web server receives latest sample and updates history buffer
`coreQueue`	1	sensor task	CoreIOT receives latest sample for periodic telemetry
`tinyMLQueue`	1	sensor task	TinyML receives latest sample stream for batching and inference
`tinyResultQueue`	1	TinyML task	LCD, web, and CoreIOT consume the latest inference result

The use of `xQueueOverwrite()` on most single-slot queues is particularly appropriate because the system is monitoring-oriented. The most recent temperature or humidity value is more

important than the complete backlog of old samples.

3.8 State Flow Through the System

Figure generation is not embedded directly inside this report, but the architecture can be reproduced externally using the following Mermaid code. This code can be pasted into draw.io through *Arrange* → *Insert* → *Advanced* → *Mermaid*, or into any Mermaid-compatible viewer.

```
flowchart LR
    DHT20[DHT20 Sensor] --> ST[Sensor Task]
    ST -->|temperature| LQ[ledQueue]
    ST -->|humidity| NQ[neoQueue]
    ST -->|latest sample| LCDQ[lcdQueue]
    ST -->|latest sample| WQ[webQueue]
    ST -->|latest sample| CQ[coreQueue]
    ST -->|latest sample| TQ[tinyMLQueue]

    LQ --> LT[LED Manager Task]
    NQ --> NT[NeoPixel Task]
    LCDQ --> LCDT[LCD Task]
    WQ --> WT[Web Server Task]
    TQ --> TT[TinyML Task]
    CQ --> CT[CoreIOT Task]

    TT -->|tinyml_result| TRQ[tinyResultQueue]
    TRQ --> LCDT
    TRQ --> WT
    TRQ --> CT

    WT -->|internetSemaphore| CT
    WT --> WIFI[AP/STA Wi-Fi Manager]
    WT --> PREFS[Preferences Storage]
    WT --> BROWSER[HTTP Dashboard]
    CT --> CLOUD[CoreIOT MQTT]
```

The diagram highlights the main design pattern of the system: one producer task feeds multiple

consumer tasks, while secondary tasks such as TinyML and Wi-Fi management enrich the data path with derived information and network events.

3.9 Design Rationale

Several implementation choices deserve emphasis:

- **Single responsibility per task.** Each task has one primary concern, which keeps the code easier to analyze and debug.
- **Latest-value semantics.** For monitoring and display tasks, the newest sample is more important than preserving every historical sample in a queue.
- **Bounded synchronization.** Critical resources are protected with explicit mutexes instead of ad-hoc timing assumptions.
- **Event-based cloud startup.** The CoreIOT task only starts MQTT-related work after the web task signals that STA connectivity is valid.
- **Batch-based inference.** TinyML is intentionally slower than raw sensing because it aggregates ten samples before making a prediction. This lowers noise and creates a more stable high-level output.

Overall, the design is a balanced soft real-time architecture. It remains straightforward to implement and maintain while still providing a clear RTOS decomposition and enough modularity for future extension.

4 Implementation

4.1 System Startup and Resource Allocation

The entry point of the application is intentionally lightweight. In `main.cpp`, the `setup()` function performs four steps:

1. initialize the serial port and I^2C bus,
2. allocate the application context,
3. create queues, mutexes, and the Internet-ready semaphore, and
4. spawn all RTOS tasks.

This startup strategy is important because it keeps initialization deterministic. The `loop()` function does not perform application logic; it simply yields with a long delay. In other words, after boot the entire system behavior is task-driven. This is a clear and appropriate RTOS pattern for the given assignment.

The application context is initialized by `init_app_shared_state()`, which sets default values such as invalid sensor readings, disconnected Wi-Fi state, and the default LCD content mode. This avoids undefined startup behavior.

4.2 Shared State Helpers

The file `global.cpp` implements helper functions to read and update the compact shared state block. The core logic is simple but useful:

- `lockSharedState()` and `unlockSharedState()` hide mutex handling details,
- `app_set_latest_sensor()` updates the latest temperature and humidity,
- `app_set_wifi_connected()` and `app_get_wifi_connected()` manage connectivity state, and
- `app_set_lcd_content_mode()` and `app_get_lcd_content_mode()` coordinate LCD display policy.

The value of this module is not its complexity but its role in improving code quality. It makes synchronization explicit and reduces the need for broad `extern` globals. This is a meaningful improvement over directly exposing mutable state to every file.

4.3 Sensor Task

The sensor task is the timing anchor of the whole application. It performs periodic DHT20 acquisition every 2 seconds using `vTaskDelayUntil()`. This API is a good design choice because it maintains a stable period relative to the previous wake time, instead of accumulating drift through repeated `delay()` calls.

The execution flow is as follows:

1. initialize the DHT20 once, under I^2C mutex protection;
2. on each period, attempt to lock the I^2C bus for up to 300 ms;
3. read temperature and humidity;

4. if the data are valid, update the shared state and dispatch the sample to all consumers;
5. if the data are invalid, propagate an invalid sample to the LCD and web server for graceful failure handling.

One of the most elegant aspects of this task is the fan-out pattern. A single physical read is turned into multiple queue writes: `lcdQueue`, `webQueue`, `coreQueue`, and `tinyMLQueue`. This means the sensor is sampled exactly once per period, while the rest of the system remains modular.

4.4 LED Manager Task

The discrete LED task is implemented as an event-driven state machine. It does not read sensors directly. Instead, it receives `led_command_t` messages through `ledQueue`. The LED task supports two categories of behavior:

- continuous blinking according to the current temperature mode, and
- short pulse sequences for exceptional events.

The temperature-to-blink mapping is implemented through `classify_temperature_mode()` and `led_half_period_ticks()`:

- cold: 5-second half-period,
- normal: 1-second half-period,
- hot: 100-ms half-period,
- error: 120-ms half-period.

The task also supports quick pulse bursts:

- Wi-Fi connected: 3 fast blinks,
- error on: 6 fast blinks,
- error clear: stop pulsing and return to off.

The strong point of this design is that the LED module is both periodic and reactive. It uses queue timeouts as implicit timing intervals, so it can either wait for a new command or toggle according to the current mode without another timer task.

4.5 NeoPixel Task

The NeoPixel task is simpler but still well designed. It blocks on `neoQueue` and updates the RGB LED whenever a new humidity command arrives. The function `humidity_to_color()` maps humidity into a continuous gradient:

- 0% to 50%: red to green,
- 51% to 100%: green to blue.

This is an improvement over a strict three-bin implementation because it provides richer visual information while still satisfying the assignment's three-level requirement. The task also supports a disable command that clears the LED immediately, which is useful for future extension or power-saving scenarios.

4.6 LCD Task

The LCD task owns the local user interface. It maintains the most recent sensor sample and the most recent TinyML result, then decides what to show based on system state. The logic has three levels:

1. in AP mode, always show the sensor screen;
2. in STA mode with normal LCD content mode, show the sensor screen;
3. in STA mode with forecast LCD content mode, show the TinyML forecast.

This conditional behavior is an excellent example of policy separation. The task itself decides *how* to render each screen, while the rest of the system decides *which content mode* is currently active.

The LCD module also reacts to environmental context:

- if the sensor data are invalid, it prints `Sensor Error`;
- if Wi-Fi is connected, it can display signal quality in percent and a qualitative label such as `GOOD` or `WEAK`;
- if TinyML results exist, it shows forecast label and rain probability.

An important implementation detail is that LCD rendering is not performed at a fixed blind period. Instead, the task updates when a new sensor sample arrives, when TinyML changes, when RSSI changes meaningfully, or when AP/STA mode changes. This avoids unnecessary

I^2C traffic and makes the display more event-aware.

4.7 Web Server Task

The web module is the largest subsystem in the repository. It combines `WebServer`, `Preferences`, local runtime state, and Wi-Fi mode management. Its implementation can be divided into five functional groups.

1) AP/STA workflow.

The board starts either from the most recently saved Wi-Fi profile or directly in AP mode if no profile exists. The helper functions `startApMode()` and `startStaConnect()` reset Wi-Fi state, configure the correct mode, and update internal status flags. If STA connection fails after 10 seconds, the system automatically returns to AP mode. The BOOT button also forces AP mode manually.

2) Persistent Wi-Fi profiles.

The project uses `Preferences` to store up to five Wi-Fi profiles. The most recently used profile is kept at index 0. The code supports:

- reading saved profiles,
- updating or inserting a profile,
- removing all saved profiles.

This is a practical improvement because it lets the board remember STA credentials across restarts without hard-coding a single network.

3) AP dashboard and settings pages.

The root page in AP mode displays:

- temperature and humidity,
- TinyML label and probability,
- AP IP and Wi-Fi status,
- target STA SSID and STA IP,
- external LED control,

- chart history, and
- quick Wi-Fi scan results.

The settings page expands the Wi-Fi management workflow by providing scan, saved-profile, and connect forms. The STA page is intentionally simpler and acts mainly as a status page rather than a full dashboard.

4) REST endpoints.

The task registers multiple routes:

- `/` for the main page,
- `/settings` for Wi-Fi configuration,
- `/api/state` for live JSON state,
- `/api/scan` for Wi-Fi scan results,
- `/api/saved` for stored Wi-Fi profiles,
- `/connect` for STA connection request,
- `/forget` for profile reset,
- `/api/light/toggle` for on/off control,
- `/api/light/brightness` for PWM brightness control.

5) STA Mode (Cloud Integration and Remote Configuration)

When the device successfully establishes a connection to a Wi-Fi network, the Web Server transitions to a specialized configuration portal. This interface is crucial for defining where and how the system's telemetry data is routed:

- **Dynamic Broker Configuration:** The STA page provides a dedicated form for customizing the communication target. Users can input the **MQTT Broker Host (Server Name)**, **Username**, and **Password** directly through the browser. This allows the system to be pointed to the official CoreIOT platform, a private MQTT broker, or a custom **Local Server** without any code changes.
- **Persistence and REST API:** Configuration changes are submitted via the `/api/coreiot/confi` POST endpoint. The task processes these requests and uses the `Preferences` library to

save the credentials into the `WIFI_STORE_NAMESPACE`, ensuring the settings persist even after a power cycle.

- **CoreIOT Task Synchronization:** The `coreiot_task` is designed to be reactive to these changes. When a new server host or set of credentials is saved, the web task updates the shared application context. The CoreIOT task, which is constantly monitoring the connection state, will automatically trigger a reconnection sequence to the newly specified server.
- **Telemetry Routing Control:** The interface also includes a toggle to enable or disable publishing. This gives the user full control over data privacy, allowing them to stop cloud transmission or redirect the stream to a local endpoint for private data logging.

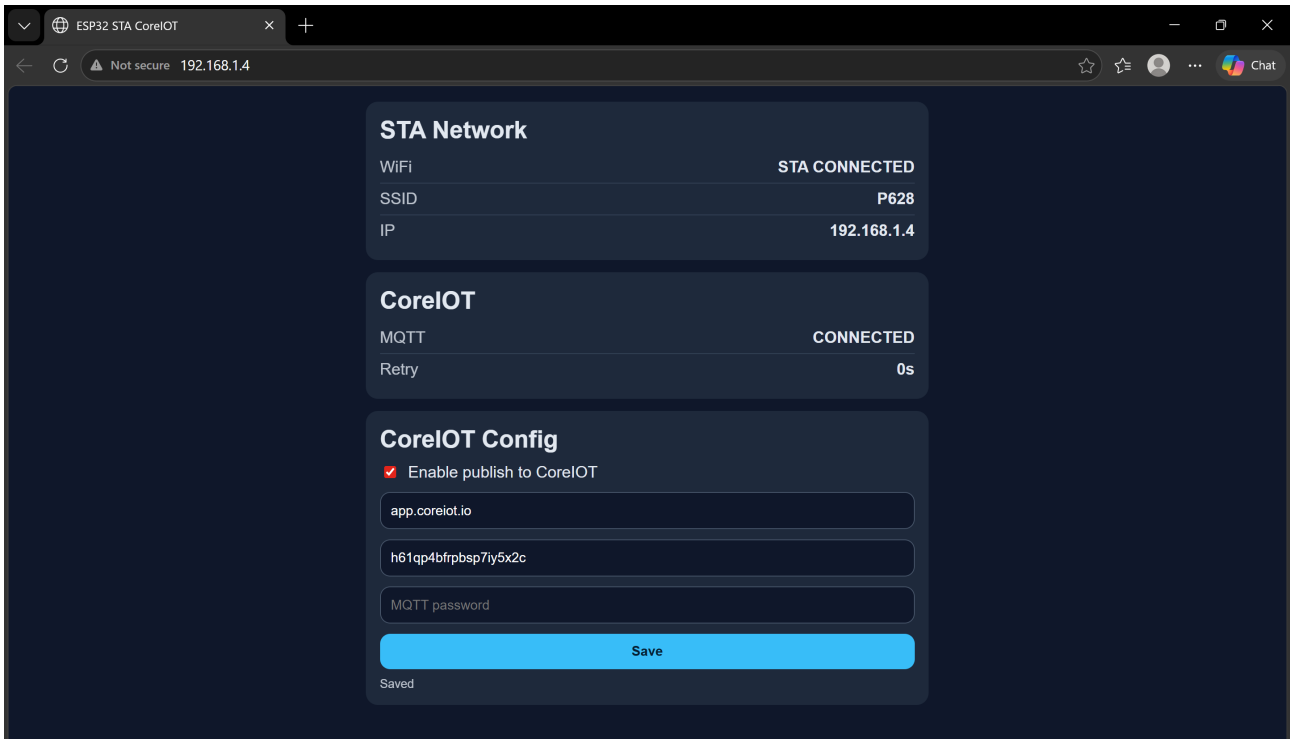

Figure 4.1: Webserver STA UI

6) Client-side visualization.

The AP dashboard includes JavaScript that polls `/api/state` every 2 seconds. It updates visible values and renders a chart on a `canvas`. A recent improvement in the code is the use of dual Y-axes for temperature and humidity, which avoids the misleading effect of drawing both series on a single implicit scale. The chart also displays axis labels and relative sample times.

Two implementation highlights stand out in this module. First, the task uses a 20 ms loop period, which is short enough for responsive HTTP handling without busy-waiting. Second,

the cloud subsystem is not started blindly; instead, the web task signals `internetSemaphore` once STA connectivity is confirmed.

4.8 TinyML Task: Edge-Based Rain Forecasting Pipeline

The TinyML task implements an on-device rain prediction system, shifting the analytical workload from the cloud directly to the edge. This approach minimizes latency, reduces network dependency, and demonstrates the viability of executing neural networks on resource-constrained microcontrollers. The development and deployment of this predictive model follow a comprehensive Machine Learning pipeline, spanning from offline dataset preparation to real-time RTOS integration.

EDA

The predictive model is built upon a historical meteorological dataset collected in Ho Chi Minh City, encompassing continuous weather observations from 2009 to 2020. The primary objective is to perform a binary classification, categorizing the upcoming weather state as either ”RAIN” (Class 1) or ”SUNNY/SAFE” (Class 0).

Dataset Overview: The dataset comprises 34,976 valid samples with zero missing values across all features. The temporal span extends from January 1, 2009, to December 20, 2020, representing a complete 4,372-day period with 100% data coverage.

Dataset Features: Table presents the complete feature structure of the meteorological dataset:

Feature Name	Data Type	Valid Samples	Missing Values
Date	datetime64[ns]	34,976	0
Time	object	34,976	0
Weather	object	34,976	0
Temp	object	34,976	0
Feels	object	34,976	0
Wind	object	34,976	0
Gust	object	34,976	0
Rain	object	34,976	0
Humidity	object	34,976	0
Cloud	object	34,976	0
Pressure	object	34,976	0
Vis	object	34,976	0

Table 4.1: Dataset Features and Data Quality Metrics

Sample Data Examples: Table displays five representative samples randomly selected from the dataset to illustrate the data characteristics and format:

Date	Time	Weather	Temp	Feels	Wind	Gust	Rain	Humidity	Cloud	Pressure	Vis
2010-10-09	12:00	Partly cloudy	31°C	39°C	6 km/h W	7 km/h	0.0 mm	72%	59%	1006 mb	Excellent
2013-11-19	03:00	Clear	23°C	26°C	8 km/h NNE	13 km/h	0.0 mm	90%	14%	1010 mb	Excellent
2016-04-22	21:00	Clear	28°C	32°C	22 km/h SE	31 km/h	0.0 mm	70%	4%	1009 mb	Excellent
2013-12-22	00:00	Clear	22°C	23°C	4 km/h N	7 km/h	0.0 mm	85%	22%	1013 mb	Excellent
2019-12-15	06:00	Partly cloudy	25°C	27°C	7 km/h NNE	9 km/h	0.0 mm	77%	9%	1012 mb	Excellent

Table 4.2: Representative Dataset Samples

Temporal Distribution Analysis: Table summarizes the temporal characteristics of the dataset:

Temporal Criterion	Value	Notes
Start Date	01/01/2009	First recorded sample
End Date	20/12/2020	Last recorded sample
Total Time Span	4,372 days	Linear period from start to end
Days with Data	4,372 days	Coverage: 100.00%

Table 4.3: Temporal Distribution Analysis

During the offline data preparation phase, the raw weather labels were grouped to form the binary target variable. Conditions containing keywords such as 'rain', 'thunder', 'drizzle', or 'storm' were mapped to Class 1, while all other conditions were mapped to Class 0. To ensure the neural network converges effectively during training, the input features were standardized using `StandardScaler`, giving the data a mean of zero and a variance of one.

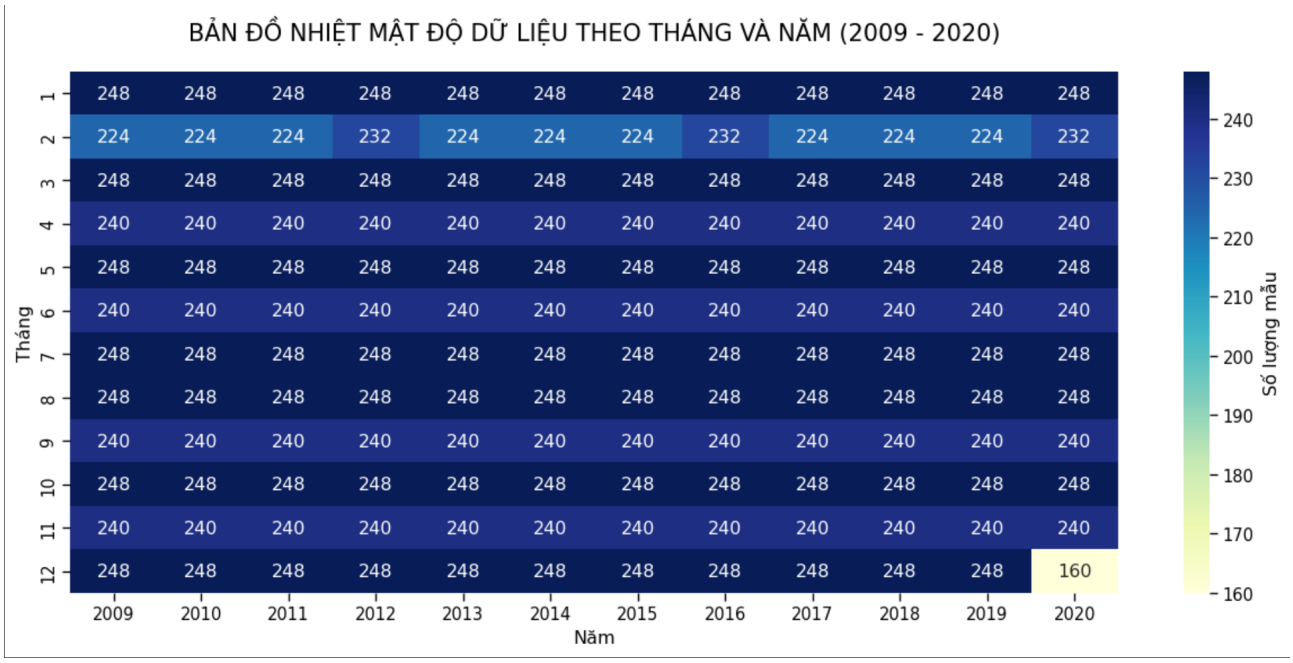

Figure 4.2: Dataset overview

Feature Engineering and Extraction

The raw dataset contains the original weather observation fields, but the model is trained on a compact numerical representation derived from `Temp`, `Humidity`, `Date`, and `Time`. The preprocessing pipeline first extracts numerical temperature and humidity values, then converts calendar information into cyclical time features, and finally constructs a feature vector that captures both the current state and short-term change patterns.

The exact feature engineering steps are implemented as follows:

1. **Temperature and humidity extraction:** `Temp_c` is extracted from the `Temp` string field, while `Hum_p` is extracted from `Humidity`. This converts the raw text measurements into numeric values usable by the neural network.

2. **Temporal encoding:** The month is mapped from the date string and the hour is parsed from the time string. Instead of using raw month and hour values directly, the model encodes them with sine and cosine functions to preserve their circular nature.

3. **Short-term deltas:** `Temp_diff` and `Hum_diff` are computed using the first-order difference between consecutive samples. These variables represent local trend changes, which are useful for capturing sudden weather transitions.
4. **Final input vector:** The model uses eight engineered features: `Temp_c`, `Hum_p`, `Temp_diff`, `Hum_diff`, `Month_sin`, `Month_cos`, `Hour_sin`, and `Hour_cos`.

This design is appropriate for the dataset because it combines instantaneous weather state with trend information and periodic time context. The scaling step using `StandardScaler` is then applied only on the training split, ensuring that all features contribute to learning on a comparable numerical scale.

Evaluation of the EDA figures. The descriptive statistics figure shows that the engineered variables are well formed and fully populated across 34,976 samples. The central values are physically plausible: temperature is concentrated around the high-20s Celsius, humidity is centered in the mid-70%, and the temporal features have means close to zero after sine-cosine encoding, which is expected for cyclical variables. The spread of `Temp_diff` and `Hum_diff` is also moderate, indicating that the dataset contains both stable periods and noticeable transitions rather than being dominated by flat readings.

The class-distribution chart shows a clear imbalance, with Class 0 accounting for 78.8% and Class 1 accounting for 21.2%. This means the problem is not perfectly balanced, so accuracy alone is not sufficient to judge model quality. The imbalance also explains why the model should be evaluated with additional measures such as precision, recall, and F1-score, especially for the rain class.

The correlation heatmap confirms that no single feature has an overwhelmingly strong linear relationship with the target label. The label correlates only weakly with the engineered inputs, which is a useful result because it suggests the rain pattern is not trivially separable by one variable alone. At the same time, the heatmap shows meaningful dependencies among features, such as the strong negative relationship between `Temp_diff` and `Hum_diff`, as well as the seasonal and daily structure captured by the cyclic time encodings. This supports the choice of a small neural network, since a nonlinear model is better suited than a simple linear classifier for combining weak but complementary signals.

1. BẢNG THỐNG KÊ MÔ TẢ ĐẶC TRƯNG

	count	mean	std	min	25%	50%	75%	max
Temp_c	34976.000000	27.845437	3.413878	16.000000	25.000000	27.000000	30.000000	39.000000
Hum_p	34976.000000	73.959858	15.677216	21.000000	64.000000	77.000000	87.000000	99.000000
Temp_diff	34976.000000	0.000057	2.553558	-8.000000	-1.000000	0.000000	1.000000	9.000000
Hum_diff	34976.000000	-0.000944	10.163728	-38.000000	-5.000000	1.000000	5.000000	35.000000
Month_sin	34976.000000	-0.004797	0.706656	-1.000000	-0.866025	-0.000000	0.500000	1.000000
Month_cos	34976.000000	-0.004550	0.707547	-1.000000	-0.866025	-0.000000	0.591506	1.000000
Hour_sin	34976.000000	0.000000	0.707117	-1.000000	-0.707107	0.000000	0.707107	1.000000
Hour_cos	34976.000000	-0.000000	0.707117	-1.000000	-0.707107	-0.000000	0.707107	1.000000

Figure 4.3: Statistical Summary of Engineered Features

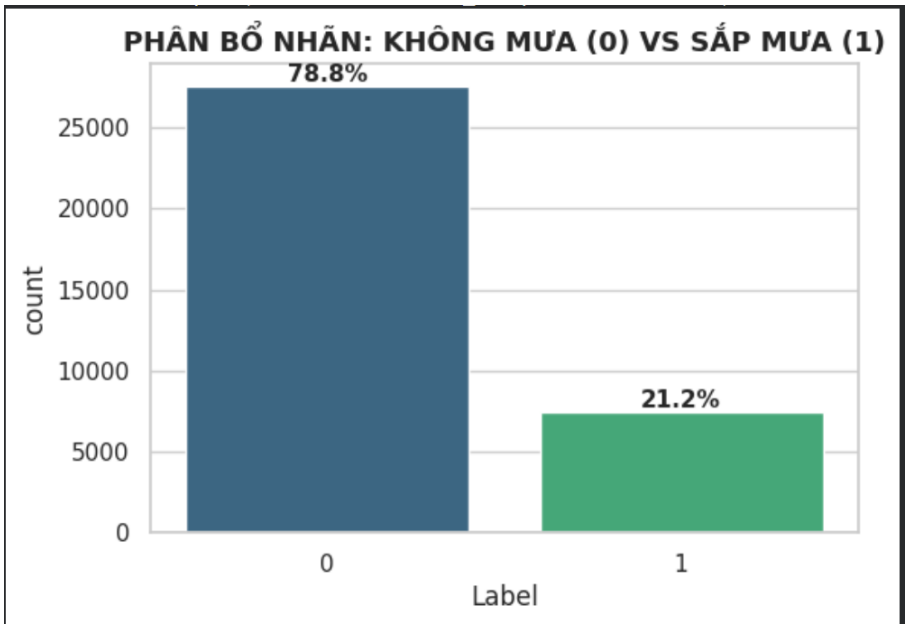

Figure 4.4: Class Distribution After Binary Labeling

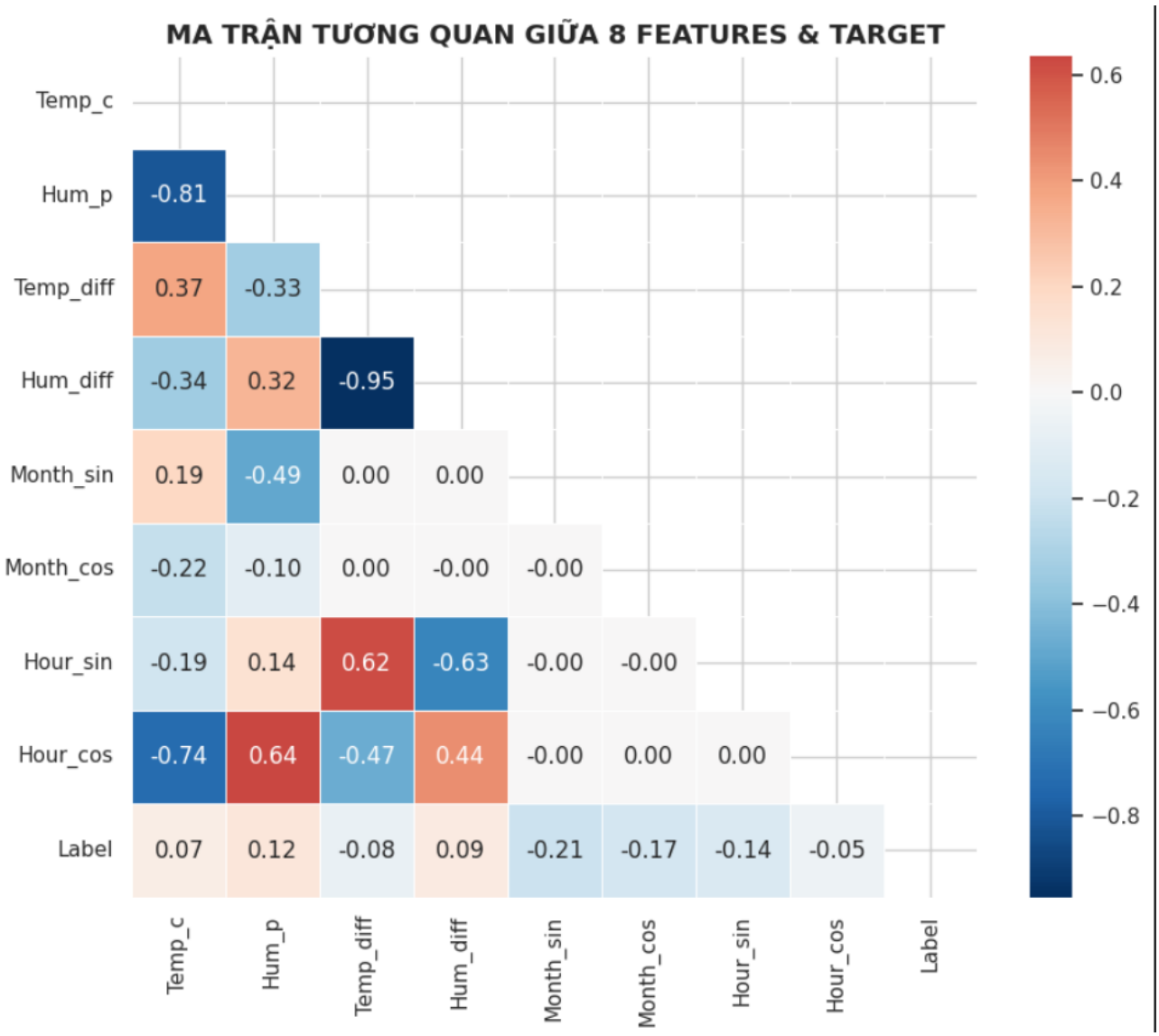

Figure 4.5: Feature Correlation Heatmap

Model Architecture and Training

Model Benchmarking and ANN Selection. Table 4.4 summarizes the candidate models evaluated on the same feature set. The results show that XGBoost achieved the highest accuracy, but the ANN was selected for the final ESP32 implementation because it provides competitive performance while remaining a compact deep learning model that can be quantized and deployed through TensorFlow Lite Micro.

The core logic is driven by a sequential Artificial Neural Network (ANN) designed via TensorFlow/Keras. Although the benchmark shows that XGBoost has a slightly higher accuracy, the ANN was intentionally chosen because the goal of this project is to demonstrate a deployable deep learning pipeline on the ESP32-S3. The ANN is lightweight enough for full integer quan-

tization, yet expressive enough to capture non-linear relationships in the engineered features. The network topology consists of:

- An **Input Layer** accepting the 8 standardized features.
- A **Hidden Dense Layer** with 12 neurons utilizing the ReLU (Rectified Linear Unit) activation function.
- A **Second Hidden Dense Layer** with 8 neurons, also using ReLU activation.
- An **Output Layer** with a single neuron applying a Sigmoid activation function, producing a final probability score between 0.0 and 1.0.

During the offline training phase, the dataset was split into an 80% training set and a 20% testing set. The model was compiled using the Adam optimizer and a binary cross-entropy loss function. After training for 50 epochs with a batch size of 32, the baseline model achieved an accuracy of **84.86%** on the 32-bit floating-point (float32) test set.

Model	Accuracy (%)	F1-Score	Parameters / Nodes	Prediction Time per Sample (ms)
XGBoost	84.33	0.5941	100	0.0048
ANN (Selected)	84.28	0.5572	221	0.1305
Decision Tree	83.73	0.5715	1,187	0.0012
KNN (k=5)	83.32	0.5792	223,840	0.0606
SVM (RBF)	82.76	0.4311	81,224	1.4149
Random Forest	81.93	0.5549	1,000,668	0.0436
Gaussian Naive Bayes	76.89	0.4079	32	0.0015

Table 4.4: Benchmark Comparison of Candidate Models

4) Full Integer Quantization for Edge Deployment

To deploy this trained model onto the ESP32-S3, it required an aggressive optimization process using the TensorFlow Lite Converter. Deploying a float32 model would consume excessive SRAM and execution time. Therefore, the model underwent **Full Integer Quantization (INT8)**.

A Representative Dataset generator (yielding 100 random training samples) was used to calibrate the dynamic range of the model's weights and activations. This calibration allowed the converter to safely map the 32-bit floats into 8-bit integers. The resulting optimized model (`.tflite`) was then converted into a C-byte array header file (`model_data.h`) using the `xxd` utility.

Remarkably, after this strict quantization process, the model retained an accuracy of **84.43%**. This corresponds to an absolute drop of only **0.43 percentage points**, or about **0.51%** relative to the original float32 accuracy. This is a highly favorable trade-off because the INT8 model drastically reduces the memory footprint and accelerates inference time on the microcontroller without utilizing floating-point hardware during the forward pass.

5) C++ Implementation in the RTOS Environment

Within the firmware, the `tiny_ml_task` manages the real-time execution of this pipeline. It utilizes the `tflite::MicroInterpreter` and an `AllOpsResolver` initialized with a 16 KB tensor arena. The implementation strictly mirrors the offline training pipeline:

- **Accumulation:** The task receives raw data from `tinyMLQueue` and accumulates 10 consecutive sensor samples (equivalent to a 20-second monitoring window) before triggering an inference cycle.
- **Normalization:** The computed 8 features are standardized locally using pre-computed `means` and `scales` arrays extracted from the Python `StandardScaler`.
- **Input Quantization:** The normalized float values are converted into the INT8 format required by the model using the input tensor's specific `scale` and `zero_point` parameters.
- **Inference and Output:** The `interpreter->Invoke()` function is called. The resulting INT8 output is de-quantized back into a float probability. If this probability exceeds the 0.5 threshold, the system categorizes the upcoming state as "RAIN".

Finally, this prediction, along with the calculated probability percentage and timestamps, is packaged into a `tinyml_result_t` structure and pushed to the `tinyResultQueue`. It is subsequently consumed by the LCD task for immediate local display and by the CoreIOT task for remote cloud telemetry.

Mathematical Formulation of the Selected ANN. Let the final engineered input vector be

$$\mathbf{x} = [x_1, x_2, \ldots, x_8]^T = [\texttt{Temp_c}, \texttt{Hum_p}, \texttt{Temp_diff}, \texttt{Hum_diff}, \texttt{Month_sin}, \texttt{Month_cos}, \texttt{Hour_sin}, \texttt{Hour_c}$$

The selected ANN transforms this input into a rain probability through two hidden layers and one sigmoid output layer:

$$\mathbf{h}_1 = \mathrm{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1),$$

$$\mathbf{h}_2 = \mathrm{ReLU}(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2),$$

$$\hat{y} = \sigma(\mathbf{W}_3\mathbf{h}_2 + b_3),$$

where $\mathrm{ReLU}(z) = \max(0, z)$ and $\sigma(z) = \frac{1}{1+e^{-z}}$. The output $\hat{y} \in [0, 1]$ represents the predicted probability of rain. The final class decision is computed as

$$\hat{c} = \begin{cases} 1, & \text{if } \hat{y} \geq 0.5, \\ 0, & \text{otherwise.} \end{cases}$$

This formulation matches the deployed ANN structure and explains how the firmware converts the 8-feature input vector into a binary weather forecast.

4.9 CoreIOT Task: Cloud Telemetry and Remote Configuration

The CoreIOT task is the cloud-integration layer of the entire embedded system. Its primary responsibility is to establish persistent connectivity with an external MQTT broker (either the CoreIOT cloud platform or a custom local MQTT server) and to bidirectionally stream telemetry data and remote control commands. The task is designed to be highly configurable, extensible, and reactive to network state changes, allowing the system to adapt to different deployment scenarios without firmware recompilation.

Initialization and Network Readiness

The CoreIOT task does not start immediately after boot. Instead, it follows a staged initialization pattern:

1. **Wait for Internet Readiness:** The task blocks on `internetSemaphore`, which is signaled by the Web Server task only after the device has successfully established Wi-Fi connectivity in STA mode.

2. **Retrieve Configuration:** Once the semaphore is released, the task retrieves the dynamic MQTT connection parameters from persistent storage (managed by `Preferences`). This includes the **broker host**, **broker port**, **username**, and **password**.

3. **Connect to Broker:** The task attempts to establish a TCP connection to the specified MQTT broker using the provided credentials.

4. **Subscribe to Control Topics:** After successful connection, the task subscribes to RPC (Remote Procedure Call) request topics, enabling the cloud dashboard to send commands back to the device.

5. **Periodic Telemetry Publishing:** Once fully initialized, the task enters a loop that publishes telemetry data every 10 seconds.

This staged approach ensures that the cloud task does not consume resources or attempt connection during AP mode, and it allows the Wi-Fi subsystem to stabilize before the cloud subsystem is activated.

Dynamic Broker Configuration and Extensibility

A key architectural feature of the CoreIOT task is its **dynamic server integration**. The MQTT broker host, port, username, and password are **not hard-coded** into the firmware. Instead, these settings are:

- **Configured via the Web Interface:** When the device connects to a Wi-Fi network (STA mode), the user can navigate to the web dashboard and access the cloud settings form. This form allows users to specify:
  - **MQTT Broker Host:** The IP address or fully qualified domain name (FQDN) of the target MQTT server. Examples:
    - For CoreIOT Cloud: `app.coreiot.io` (default if not overridden).
    - For a local MQTT broker running on a desktop computer in the same network: `192.168.1.100`.
  - **MQTT Broker Port:** Typically `1883` for standard MQTT (or `8883` for MQTT over TLS).
  - **Username:** For CoreIOT cloud, this is the **access token** provided by the CoreIOT platform (e.g., `srQf3iXbZDdZjL8PuatC`). For a local broker, this can be any username configured in the broker's authentication system.
  - **Password:** For CoreIOT cloud, the password field can be left empty or set to a placeholder (CoreIOT uses token-based auth). For a local MQTT broker with user/password authentication, this is the password corresponding to the configured username.
- **Persisted Across Restarts:** Once configured, these settings are stored in the `Preferences` storage under the `WIFI_STORE_NAMESPACE`. They survive power cycles and Wi-Fi reconnections.

- **Reactive Reconnection:** If the user modifies the broker settings via the web interface, the CoreIOT task automatically detects the change. Upon the next configuration check (or upon reconnection), the task will disconnect from the old broker and connect to the newly specified broker.

This design allows seamless deployment flexibility:

- In a **cloud-connected scenario**, the device publishes telemetry to the official CoreIOT platform (`app.coreiot.io`) using the CoreIOT access token.
- In a **private/local deployment scenario**, the device can be reconfigured via the web form to point to a local MQTT broker running on a private server, without any code changes or recompilation.

Telemetry Publishing and Payload Structure

The CoreIOT task publishes rich, structured telemetry data every 10 seconds. The payload includes:

- **Temperature (°C):** Real-time sensor reading from the DHT20.
- **Humidity (%):** Real-time sensor reading from the DHT20.
- **Timestamp:** Unix epoch time (if NTP sync is available) or a tick-based fallback for relative timing. This enables accurate time-series data in the cloud dashboard.
- **Rain Probability (0-1):** Continuous probability output from the quantized TinyML model, ranging from 0.0 (no rain) to 1.0 (certain rain).
- **Rain Probability Percentage (0-100):** User-friendly percentage representation of the probability.
- **Categorical Rain Prediction:** Boolean classification (”RAIN” or ”SUNNY”) determined by the 0.5 threshold applied to the probability.
- **Short Rain Status String:** Human-readable forecast label such as `"Rain Expected"` or `"Safe Weather"`.
- **Current LCD Content Mode:** Indicates whether the local display is showing real-time sensor data or TinyML forecast. This can be controlled remotely.
- **Wi-Fi Connection State:** Signal strength (RSSI) and connection status, allowing the cloud dashboard to monitor device connectivity health.

This payload structure is carefully chosen because it balances data richness with network efficiency. Rather than sending only raw temperature and humidity, the system also transmits:

- Derived analytics (TinyML prediction and probability),
- Contextual metadata (timestamp, display mode, signal strength), and
- Redundant representations (both 0-1 probability and 0-100 percentage).

This redundancy ensures that cloud dashboards and local logging systems have full context without requiring post-processing.

Remote Control and RPC Interaction

The CoreIOT task is not purely a data publisher; it also acts as an **RPC responder**. The task subscribes to RPC request topics and can accept commands such as:

- **Display Mode Toggle:** A cloud-side widget can send a command to switch the local LCD between real-time sensor display and TinyML forecast display.
- **Parameter Query:** Commands to request the current state, configuration, or other device metadata.
- **Control Signals:** Future extensions could include commands to adjust sampling intervals, change broadcast thresholds, or trigger firmware updates.

The RPC parser is designed to be flexible, accepting various boolean-like parameter formats (e.g., `"true"`, `"1"`, `"on"`) and mapping them to the appropriate local action. This approach enables a responsive, bidirectional cloud-to-device communication pattern without imposing a rigid command schema.

Failure Handling and Reconnection Logic

The CoreIOT task implements robust failure recovery:

- **Connection Loss Detection:** If the MQTT broker becomes unreachable, the task detects the disconnection and enters a reconnection loop.
- **Exponential Backoff:** Rather than attempting immediate reconnection (which could overwhelm the network), the task implements an exponential backoff strategy, with configurable retry intervals.

- **Graceful Degradation:** If cloud connection is unavailable, the local system continues to operate normally. Telemetry is not published, but sensor readings, LCD display, and local features remain fully functional.
- **Automatic Recovery:** Once the broker becomes reachable again, the task re-establishes connection and resumes publishing without manual intervention.

This design ensures that network transients do not compromise the integrity of the entire embedded system.

4.10 MQTT Broker Implementation: TinyMQTT Service

The MQTT broker is a standalone Python service designed to serve as an intermediary between the ESP32 IoT devices and the CoreIOT cloud platform. Unlike the firmware-side CoreIOT task (which acts as a client), the broker is a stateful service that accumulates telemetry from multiple devices, aggregates their data, and periodically forwards the aggregated stream to CoreIOT. The implementation demonstrates a complete IoT gateway pattern suitable for local deployment and demonstrates flexible authentication.

Architecture and Role

The TinyMQTT broker serves three primary roles:

1. **Local MQTT Broker:** It accepts incoming connections from multiple ESP32 devices using the standard MQTT protocol. Each device is authenticated using configured username and password credentials.
2. **Data Aggregator:** It receives telemetry messages from each device, parses the JSON payloads, and stores them in an in-memory database indexed by device ID. This allows the broker to maintain a running history of samples.
3. **Cloud Gateway:** It acts as an MQTT client to the CoreIOT platform, batching up accumulated telemetry and forwarding it at regular intervals (default: every 5 seconds) to `v1/gateway/telemetry` topic using a single authenticated connection.

This three-layer pattern decouples the device communication (local MQTT) from the cloud platform communication (separate MQTT client), reducing latency on the device side and enabling flexible cloud platform switching without device code changes.

Configuration and Credentials

The broker is configured through static settings at the top of `TinyMQTT.py`:

- **BROKER_IP:** The IP address where the broker listens for incoming device connections. Default: `192.168.137.1` (adjustable based on the host machine's network interface).
- **BROKER_PORT:** The MQTT port, typically `1883` for standard MQTT.
- **BROKER_USER and BROKER_PASS:** Static credentials that all devices must use when connecting to the broker. These are hashed using SHA256 and stored in `password_file.txt`.
- **GATEWAY_HOST, GATEWAY_PORT, GATEWAY_ACCESS_TOKEN:** Configuration for the CoreIOT cloud platform. The broker connects to `app.coreiot.io:1883` and authenticates with the provided access token. This token must be obtained from the CoreIOT admin dashboard.
- **GATEWAY_PUBLISH_INTERVAL_SEC:** The interval (default 5 seconds) at which the broker batches and sends telemetry to CoreIOT.

All configuration is done at startup; no dynamic reconfiguration is currently supported, allowing the broker to be a simple, stateless service that can be containerized or deployed to a dedicated machine.

Device Auto-Detection and Dynamic Device Management

Unlike pre-configured systems that require manual device registration, the TinyMQTT broker implements **automatic device discovery**:

- **Topic-Based Device Identification:** Each incoming MQTT message is analyzed by its topic path. The broker expects messages published to topics of the form `devices/<device_id>/telemetry`. The second segment of the topic is extracted as the **device ID**.
- **JSON Payload Parsing:** Each message is expected to be a JSON object containing sensor data. If the JSON includes a `device_id` field, that value is used; otherwise, the device ID from the topic is retained.
- **Dynamic Device Creation:** The first time a message arrives from a new device ID, the broker automatically creates an entry for that device in the in-memory database. No manual registration, configuration file, or administrative action is required. Subsequent messages from the same device ID are appended to its growing sample list.

This approach offers several advantages:

- **Zero Setup:** New devices can join the system by simply connecting to the broker and publishing to the standard topic pattern. They appear in the system immediately.
- **Scalability:** The broker can handle dozens or hundreds of devices without pre-configuration.
- **Flexibility:** Devices can dynamically change their device ID (though this is not recommended in practice) by publishing to a different topic path.

In-Memory Data Storage and Buffering

The broker uses two key data structures to manage incoming telemetry:

- **DEVICE_SAMPLES:** A dictionary mapping device ID to a list of samples. Each sample is a dictionary containing `temperature`, `humidity`, `rain_prob`, and `ts` (timestamp) fields. This structure persists indefinitely, maintaining a complete historical record of all samples received during the broker's uptime. It serves as a local data archive.
- **DEVICE_PENDING:** A secondary dictionary also mapping device ID to samples. However, this list is cleared every time the gateway publisher sends telemetry to CoreIOT. Thus, DEVICE_PENDING represents the "pending batch" of samples waiting to be forwarded to the cloud. This two-buffer pattern ensures that:
  - Local archival is not lost when syncing to the cloud.
  - The cloud publisher sends only new samples, avoiding redundant uploads.

Both dictionaries are protected by `DEVICE_LOCK`, a threading.Lock() object, ensuring thread-safe access when the subscriber thread is receiving messages and the gateway publisher thread is reading and clearing the pending buffer simultaneously.

Message Reception and Storage Workflow

The on-device message reception is handled by the `run_subscriber()` function, which runs in a dedicated thread:

1. **Connection Establishment:** The subscriber creates an MQTT client, authenticates with `BROKER_USER` and `BROKER_PASS`, and connects to the local broker (127.0.0.1) at the configured port.

2. **Topic Subscription:** The subscriber subscribes to the wildcard topic `#`, which matches all incoming messages published to the broker.
3. **Message Reception:** Upon receiving each message, the `on_message()` callback is triggered. The payload is decoded as UTF-8 and parsed as JSON.
4. **Device ID Extraction:** The topic path is split. If it matches the pattern `devices/<device_id>/telemetry`, the device ID is extracted from the second segment. Otherwise, it attempts to read the device ID from the JSON `device_id` field.
5. **Sample Construction:** A dictionary containing the relevant fields (`temperature`, `humidity`, `rain_prob`, `ts`) is built from the JSON payload.
6. **Thread-Safe Storage:** Under mutex protection, the sample is appended to both `DEVICE_SAMPLES[device_id]` and `DEVICE_PENDING[device_id]`. If either list does not yet exist, it is created.
7. **Logging:** A console message is printed indicating the device ID, total sample count, and the stored sample content. This provides real-time visibility into the broker's data ingestion.

This workflow ensures that:

- Every sample from every device is captured, parsed, and stored.
- Invalid JSON is gracefully rejected (with a warning message).
- Non-JSON payloads are also logged and skipped.
- Device IDs are dynamically detected and managed without pre-registration.

Batched Cloud Publishing to CoreIOT

In parallel to the subscriber thread, the `start_gateway_publisher()` function runs in a separate thread and implements the cloud synchronization logic:

1. **Cloud Connection:** The publisher creates a second MQTT client, authenticates using the `GATEWAY_ACCESS_TOKEN` as the username (CoreIOT uses token-based auth), and connects to `app.coreiot.io` at port 1883.
2. **Periodic Batching:** Every `GATEWAY_PUBLISH_INTERVAL_SEC` seconds (default 5 seconds), the publisher wakes up and checks if there are any pending samples.

3. **Snapshot and Clear:** Under mutex protection, the publisher captures a snapshot of the current `DEVICE_PENDING` dictionary and immediately clears it. This ensures that:
   - Even if new samples arrive during the upload, they are not lost.
   - The next batch will only contain samples received after the previous batch.
4. **Payload Construction:** For each device in the snapshot, the publisher iterates through its pending samples. For each sample:
   - If the sample includes a `ts` field, it is converted to milliseconds (CoreIOT expects millisecond precision).
   - If `ts` is missing, the current system time is used.
   - A dictionary containing `ts` and `values` (temperature, humidity, rain_prob) is appended to a per-device list.
5. **JSON Serialization and Upload:** The entire payload structure is serialized to JSON and published to the `v1/gateway/telemetry` topic on CoreIOT. The CoreIOT platform then deserializes this telemetry and associates each sample with the corresponding device ID.
6. **Logging:** Upon successful publish, a message is logged indicating successful gateway delivery.

The resulting payload structure sent to CoreIOT has the following form:

```
{
  "device_1": [
    {"ts": 1700000000000, "values": {"temperature": 25.3, "humidity": 60, "rain_prob"
    {"ts": 1700000005000, "values": {"temperature": 25.5, "humidity": 61, "rain_prob"
  ],
  "device_2": [
    {"ts": 1700000001000, "values": {"temperature": 28.1, "humidity": 55, "rain_prob"
  ]
}
```

This structure enables CoreIOT to:

- Track multiple devices in a single batch.
- Associate each sample with precise timestamps.

- Build time-series charts and analytics per device.
- Apply historical analysis and anomaly detection at the cloud level.

Multi-Threading and Synchronization

The TinyMQTT broker is a multi-threaded application with three main threads:

1. **Broker Thread:** Runs the hbmqtt broker engine, which accepts device connections and routes messages internally.
2. **Subscriber Thread:** Runs the MQTT subscriber client, which listens to all messages on the broker and populates the in-memory data structures.
3. **Gateway Publisher Thread:** Periodically batches and uploads telemetry to the CoreIOT cloud.

All threads are daemon threads, meaning if the main process exits, they are terminated immediately without a graceful shutdown sequence. Thread safety is ensured through `DEVICE_LOCK` protecting all access to `DEVICE_SAMPLES` and `DEVICE_PENDING`.

A careful reading of the code shows that the subscriber and publisher never simultaneously read and write to the same structures:

- The **subscriber** continuously appends to both `DEVICE_SAMPLES` and `DEVICE_PENDING`.
- The **publisher** reads `DEVICE_PENDING`, then clears it. It never modifies `DEVICE_SAMPLES`.

This asymmetry means contention is minimized and deadlock risk is low. The lock is typically held for only a few microseconds per access.

Failure Modes and Robustness

The broker implementation exhibits several desirable failure behaviors:

- **Device Connection Loss:** If an ESP32 device disconnects, the broker continues operating. Reconnecting devices are treated as new clients and begin publishing immediately. No special recovery logic is needed.
- **Cloud Connectivity Loss:** If CoreIOT becomes unreachable, the subscriber continues to accept and buffer device messages. Samples accumulate in `DEVICE_PENDING` until cloud connectivity is restored. Once the cloud connection recovers, buffered samples are sent in the next batch cycle.

- **JSON Parse Errors:** If a device publishes malformed JSON, the subscriber logs the error and skips the message. The broker remains operational.
- **Broker Restart:** If the broker service is restarted, in-memory buffers are lost (DEVICE_SAMPLES and DEVICE_PENDING are cleared). However, once devices reconnect and resume publishing, the system returns to normal operation. For permanent archival, an external database could be integrated.

This robustness makes the broker suitable for long-running deployments in environments with occasional network transients.

Extensibility and Future Enhancements

The current TinyMQTT implementation is intentionally kept simple and modular. It is well-suited for future enhancements:

- **Database Persistence:** Adding a SQL database (SQLite, PostgreSQL) to persist DEVICE_SAMPLES beyond broker uptime.
- **Multiple Cloud Backends:** Supporting secondary cloud platforms (e.g., ThingsBoard, Azure IoT Hub) by instantiating additional gateway publisher clients.
- **Dynamic Configuration:** Implementing a REST API or web interface to reconfigure broker settings without restarting.
- **Rate Limiting and Quotas:** Adding per-device or per-message rate limits to prevent misbehaving devices from overwhelming the system.
- **Data Transformation Pipelines:** Inserting intermediate processing (e.g., anomaly detection, data aggregation) between ingestion and cloud publish.
- **MQTT Advanced Features:** Implementing QoS 2 (exactly-once delivery), retained messages, or last-will-and-testament (LWT) for enhanced reliability.

4.11 Time Utility Module

The file `app_time_utils.cpp` provides helper routines for:

- validating epoch time,
- converting epoch time into local date-time fields,
- synchronizing with NTP, and

- formatting time strings.

Although the current runtime path does not actively show a complete NTP setup sequence, this module is still important because both the TinyML and CoreIOT parts are time-aware. It creates a clean extension point for future work in which telemetry timestamps and time-derived inference features are backed by live synchronized clock values.

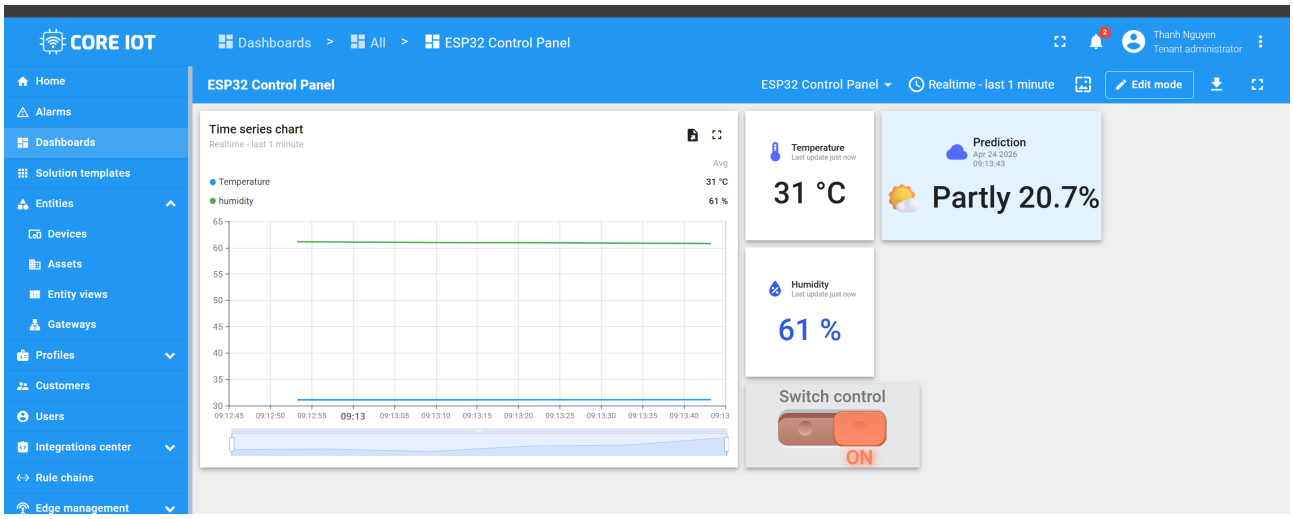

Figure 4.6: coreIOT dashboard

4.12 Implementation Qualities

Several implementation features are worth highlighting as strengths of the project:

- The system uses `xQueueOverwrite()` effectively for latest-value semantics.
- The sensing period is stabilized through `vTaskDelayUntil()`.
- Shared state is reduced and protected by a mutex.
- The LCD behavior is policy-driven rather than hard-coded to one screen.
- The web interface is not a static page; it actively supports monitoring, configuration, and control.
- The TinyML task is integrated into the same RTOS pipeline as the other system functions.
- The CoreIOT task supports both telemetry and remote RPC interaction.

Taken together, these implementation choices show that the system is not merely feature-complete for the assignment; it is architecturally coherent.

5 Testing

5.1 Testing Methodology

The repository contains extensive diagnostic logging and clearly defined RTOS periods, but it does not contain a full benchmark harness, oscilloscope traces, or deadline-monitor instrumentation. Therefore, the validation in this report is based on two evidence sources:

1. **code-driven timing analysis**, derived from task periods, queue usage, and blocking behavior; and
2. **serial-log validation**, derived from the diagnostic messages emitted by the firmware such as `[Sensor]`, `[LCD]`, `[Web]`, `[TinyML]`, and `[CoreIoT]`.

This means the project can be evaluated confidently as a *soft real-time* embedded system, but not yet as a formally verified hard real-time system.

5.2 Test Plan

The implemented test plan covers all assignment tasks and the interactions between them:

- verify periodic sensor acquisition and DHT20 initialization;
- verify temperature-to-LED blink mapping;
- verify humidity-to-NeoPixel color behavior;
- verify LCD rendering under normal, AP, STA, and TinyML states;
- verify AP dashboard response and external LED control;
- verify Wi-Fi scan, profile save, STA connect, timeout fallback, and BOOT-button AP recovery;
- verify TinyML batching and prediction publication;
- verify CoreIOT connection, RPC subscription, and telemetry publishing.

5.3 Representative Log Evidence

The following diagnostic messages are important for system-level testing:

- `[Sensor] DHT20 initialized.`
- `[Sensor] T=…, H=…`

- `[LCD] LCD initialized.`
- `[Web] AP mode started …`
- `[Web] STA connect start …`
- `[Web] STA connected …`
- `[Web] STA connect timeout -> back to AP`
- TinyML logs containing averaged values and rain probability
- `[CoreIoT] Attempting MQTT connection…`
- `[CoreIoT] Subscribe v1/devices/me/rpc/request/+ -> OK`
- `[CoreIoT] Publish -> OK`

Even without an external benchmark framework, these logs give a clear observation path for validating control-flow transitions and subsystem activation.

5.4 Timing Analysis by Task

Table 5.1 summarizes the expected timing behavior of the main tasks based on the implemented code.

5.5 Evaluation of RTOS Timing Quality

From a real-time perspective, the strongest part of the implementation is the sensing pipeline. The sensor task is explicitly periodic, and all downstream tasks are primarily event-driven. This is a good architecture because the periodic source is stable while the rest of the system reacts to the arrival of new data.

The following observations can be made:

- The project does *not* rely on a long blocking loop that mixes all concerns together.
- The sensor task has the clearest timing contract in the whole system.
- The web task remains responsive because it executes at a short 20 ms period.
- The cloud task is decoupled from sensing and therefore should not delay the DHT20 pipeline.
- The LCD and TinyML tasks trade immediacy for better stateful behavior.

Table 5.1: Timing-oriented analysis of the implemented tasks

Task	**Expected timing**	**Observation and interpretation**
Sensor task	2 s fixed period	Uses `vTaskDelayUntil()`, which is the strongest timing primitive in the project. This should keep long-term drift low as long as execution time stays below the period.
LED task	event-driven, mode-dependent half-period	Responds immediately to queue input because `xQueueReceive()` wakes as soon as a command arrives. Blink cadence depends on the current mode.
NeoPixel task	event-driven	Updates on each humidity command. Worst-case refresh is bounded by the sensor period, because humidity changes originate from the sensor task.
LCD task	event-driven, queue wait up to 250 ms	Typical update occurs soon after a new sample arrives. Additional updates occur on Wi-Fi RSSI or display-state changes.
Web task	20 ms service loop	Appropriate for responsive local HTTP handling. Human interaction latency should feel near-instant on a local AP.
TinyML task	one inference per 10 samples	Since sensing is 2 s periodic, inference is generated approximately every 20 s. This is the main intentional latency in the system.
CoreIOT task	10 s publish period	Telemetry is sent every 10 s after Wi-Fi and MQTT readiness. Network latency is non-deterministic and outside RTOS control.

However, it is also important to state what is *not* yet proven:

- Worst-case execution time (WCET) has not been measured for every task.
- There is no deadline-monitor task or explicit miss counter.
- Cloud and Wi-Fi latencies remain network-dependent.
- TinyML inference time is not timestamped in the logs as a numeric micro-benchmark.

Therefore, the system can be judged as meeting the assignment's practical RTOS expectation, but it should be classified as a soft real-time implementation rather than a formally validated hard real-time design.

5.6 Task-by-Task Functional Verification

Task 1: LED behavior.

The LED task meets the requirement of at least three temperature-dependent behaviors. The classification function distinguishes cold, normal, and hot regions, while the manager task converts each region into a visibly different blink rate. In addition, the module already includes event pulses, which improves debuggability and future extensibility.

Task 2: NeoPixel behavior.

The NeoPixel task satisfies the humidity mapping requirement and goes beyond it by providing a continuous gradient rather than only three static colors. Because the task receives data through a queue, synchronization with the sensor task is clean and non-blocking.

Task 3: LCD monitoring.

The LCD subsystem clearly satisfies the requirement of multiple display states. It supports sensor error feedback, AP/STA context awareness, and TinyML forecast display. The use of an I^2C mutex is technically important and directly aligned with the synchronization emphasis of the assignment.

Task 4: Web server.

The web task strongly satisfies the assignment. It provides:

- Wi-Fi scan,
- Wi-Fi save and recall,

- connect and fallback flow,
- an external LED on/off button,
- an external LED brightness slider,
- live sensor data display, and
- a recent-history chart.

The implementation therefore exceeds the minimum requirement of "two buttons" by including a meaningful monitoring dashboard and configuration workflow.

Task 5: TinyML.

The TinyML module fulfills the requirement of deploying a model on the device and exposing the result to the rest of the system. The current evaluation is primarily architectural and functional rather than dataset-statistical, because the repository focuses on deployment code. Still, the task is fully integrated and produces live inference output in the running firmware.

Task 6: CoreIOT.

The CoreIOT task satisfies the central cloud requirement: publishing occurs in STA mode after Internet readiness is signaled. The task also subscribes to RPC messages, which is a strong implementation detail because it turns cloud communication into a bidirectional control channel.

5.7 End-to-End Responsiveness

Looking at the end-to-end flow, the expected delays are as follows:

- **Sensor to LED/NeoPixel:** effectively one sensor period plus immediate queue wake-up, so around the 2 s sensing cadence.
- **Sensor to LCD:** typically within the same sampling cycle, plus a small scheduling/rendering delay.
- **Sensor to Web dashboard:** one sensor period plus the browser's 2 s polling interval.
- **Sensor to TinyML result:** approximately 20 s because of batching.
- **Sensor to CoreIOT publish:** bounded by the 10 s publish period, assuming Wi-Fi and MQTT are already connected.

These values are consistent with the intended operating scope of the system. The implementation is optimized for functional integration and predictable behavior rather than ultra-low-latency inference.

5.8 Current Limitations of the Test Evidence

The current code base supports convincing functional testing, but a more advanced evaluation could still improve the report:

- add explicit timestamped logs for task start/end times,
- measure TinyML inference duration directly,
- collect repeated Wi-Fi reconnect statistics,
- record actual CoreIOT publish success rates over long runs,
- export charts and screenshots from the AP dashboard.

Even without these additions, the current implementation provides convincing evidence that all major assignment tasks have been achieved and that the RTOS design behaves consistently with its intended timing model.

6 Results and Discussion

6.1 What Works Well

Several aspects of the system are already strong from both an embedded-systems and a software-engineering perspective.

First, the task architecture is coherent. The project does not implement assignment features as unrelated code fragments; instead, all major functions fit into a single RTOS dataflow model. The sensor task is the primary producer, the UI tasks are consumers, the TinyML task is a transformer, and the CoreIOT task is the cloud sink. This architectural clarity is a major strength of the system.

Second, the use of synchronization primitives is appropriate. Queues are used for data movement, mutexes are used for genuine shared-resource protection, and the binary semaphore is used for a one-time readiness signal. This is a better design than trying to replace all state with semaphores or relying on broad global-variable access.

Third, the AP dashboard is practically useful. It is not limited to a single HTML page with one button. It offers Wi-Fi scanning, profile management, live telemetry, LED control, brightness control, and recent-history visualization. This gives the project a user-facing completeness that many small embedded projects do not reach.

Fourth, the project demonstrates good feature integration. TinyML is not isolated from the rest of the system; it affects the LCD, the dashboard, and the cloud payload. Likewise, CoreIOT is not treated as a one-way uploader only; it also influences local behavior through RPC.

6.2 Tradeoffs in the Current Design

Although the implementation is successful overall, the design includes several explicit tradeoffs.

AP usability versus STA cloud publishing.

The same board cannot simultaneously behave as a simple AP-only configuration node and as a pure Internet-connected cloud device without state transitions. The chosen design resolves this by prioritizing AP mode for local setup and STA mode for cloud publishing. This is practical, but it introduces mode-switching complexity and makes some functionality context-dependent.

Fresh data versus full history.

The queues mostly use single-slot "latest value" semantics. This is the right decision for UI responsiveness and low memory consumption, but it means the system does not preserve a complete time series for every consumer. The web task therefore implements its own history buffer for visualization.

Prediction stability versus latency.

The TinyML task batches ten sensor samples before inference. This reduces noise and gives the model smoother features, but it increases prediction latency to roughly 20 seconds. For a weather-oriented embedded node, this is acceptable, but for a faster reactive use case it would be too slow.

Reduced global state versus local module state.

The project has reduced shared global variables significantly by introducing an application context and protected shared state. However, some file-local static variables still exist, especially

inside the web module. This is not a design error; it is a tradeoff between strong encapsulation and total elimination of all persistent module state.

6.3 Current Weaknesses

Several limitations remain in the current system.

Timing validation is mostly analytical.

The code is structured well for soft real-time execution, but hard timing evidence is limited. There are no per-task runtime statistics, no deadline miss counters, and no direct WCET measurements.

Cloud connectivity still depends on external network conditions.

The CoreIOT task is correctly gated by STA readiness, but MQTT success still depends on Wi-Fi quality, broker availability, and network environment. This is expected in practice, but it prevents deterministic end-to-end guarantees.

TinyML evaluation is deployment-focused.

The code clearly deploys and executes the model, but the repository itself does not yet contain a complete experimental section with confusion matrices, test sets, or repeated on-device accuracy runs. That evaluation should be added in the final report using training and test results from the model-development pipeline.

Time synchronization is prepared but not fully integrated.

The time utility module exists, but active NTP synchronization is not visible in the main execution flow. As a result, some time-derived TinyML features may rely on fallback constants and some telemetry timestamps may fall back to RTOS tick time.

The web chart is intentionally lightweight.

The dashboard chart fulfills the current monitoring requirement, but it is not yet a full monitoring frontend. It keeps only a short recent history and does not persist records across resets or support export.

6.4 Future Development Directions

The current system already satisfies the assignment, but several extensions are natural next steps:

- add explicit runtime profiling for each task,
- integrate NTP synchronization into the STA workflow,
- increase the length and persistence of the web history buffer,
- support offline buffering and delayed CoreIOT upload,
- extend cloud RPC to control more local features,
- add more sensors or actuators using the same queue-based architecture,
- improve TinyML evaluation with clearer dataset and accuracy reporting.

In summary, the system is already well structured and functionally complete, but the strongest next improvements would focus on measurement rigor, connectivity robustness, and richer data analysis.

7 Conclusion and Future Work

7.1 Conclusion

This project successfully transforms an RTOS starter code base into a complete ESP32-S3 IoT system that combines sensing, local feedback, web interaction, TinyML inference, and cloud telemetry. From the perspective of assignment coverage, all six required tasks are addressed in a coherent way rather than as independent features. Temperature is mapped to LED blink patterns, humidity is mapped to NeoPixel color, the LCD provides multiple display states, the AP web server supports both monitoring and control, the TinyML model runs on-device, and CoreIOT publishing is handled correctly in STA mode.

From the perspective of system design, the most important achievement is the task-based architecture. The application is structured around clear RTOS responsibilities, explicit queue-based communication, and proper use of semaphores and mutexes. This makes the system easier to understand, extend, and test. The introduction of an application context and protected shared state also improves code quality by reducing uncontrolled global-variable usage.

The project also provides meaningful engineering lessons. It shows how a single embedded node

can support multiple layers of functionality at once: physical sensing, local actuation, browser-based configuration, machine-learning inference, and cloud communication. It also highlights the importance of making tradeoffs explicit. For example, the system favors batching for TinyML stability, latest-value queues for simplicity, and AP/STA mode switching for practical usability.

7.2 What Was Learned

Through the development of this system, the following technical lessons were especially important:

- how to decompose a real embedded application into RTOS tasks with different priorities and activation models;
- how to use queues, semaphores, and mutexes for their proper purposes rather than treating them as interchangeable mechanisms;
- how to design a user-facing AP dashboard that remains connected to live firmware state;
- how to integrate TinyML into a streaming sensor pipeline on a resource- constrained microcontroller;
- how cloud connectivity changes the state machine of an embedded node, especially when AP and STA modes must coexist.

7.3 Future Work

Although the implemented system already meets the project goals, there are several clear paths for improvement:

- improve long-term timing analysis with explicit task profiling and deadline measurements;
- integrate NTP synchronization directly into the STA connection path;
- evaluate the TinyML model with a larger on-device test set and report formal accuracy metrics;
- extend CoreIOT interaction with additional RPC-controlled features;
- store longer-term sensor history and expose it through the web interface;
- add recovery features such as offline telemetry buffering during temporary Wi-Fi loss.

Overall, the implemented system shows that an ESP32-S3 platform can host a well-structured and feature-rich IoT application when its software is designed around explicit RTOS principles rather than ad-hoc control logic.